Security Architecture for AI Coding Agent Harnesses

First Author[1], Second Author[2], Third Author[2], Fourth Author[1] and Fifth Author[1]
[1]Pragnition Labs, [*]Equal Contribution, [1]Affiliation 1, [2]Affiliation 2

AI coding agents operate with developer-level credentials, yet the security architectures governing them remain largely ad hoc. A compromised or misbehaving agent has the same blast radius as a compromised developer: access to source code, secrets, CI pipelines, and production infrastructure. The difference is that the developer exercises judgment; the agent follows instructions that may be adversarially manipulated. We present a formal framework for security architecture in AI coding agent harnesses, grounding it in four established bodies of theory: (1) access control theory, formalizing the sandbox as a capability restriction operator on the agent's action space, with the HRU undecidability result bounding what can be proven about general sandbox configurations; (2) information flow control, modeling credential management as a lattice-based flow problem where noninterference guarantees that secrets above the agent's clearance never enter its observable state; (3) the structural enforcement principle, proving that prompt-based security compliance decays as $(1-\epsilon)^T$ over T interaction steps while structural enforcement does not decay; and (4) adversarial robustness analysis, connecting prompt injection to the confused deputy problem and establishing, via the BEB impossibility framework, that alignment-based defenses cannot eliminate injection vulnerabilities. We calibrate these models against empirical data from six production harnesses (Codex, Claude Code, Cursor, Devin, Spotify Honk, RAPID) and five independent measurement studies (Apollo Research, NIST CAISI, METR, CodeRabbit, AgentSpec). The framework identifies a convergent four-layer reference architecture (execution isolation, credential management, output filtering, adversarial input isolation) and derives ten design principles. Five contrarian positions are engaged with evidence rather than dismissed. The central result is architectural: the agent is outside the Trusted Computing Base by design, and security reduces to the correctness of a small, non-AI, formally verifiable enforcement boundary.

1. Introduction

AI coding agents now generate a majority of committed code on major platforms, with GitHub reporting that Copilot produces over 50% of code in files where it is active. These agents operate with developer credentials: git tokens, API keys, shell access, and (in some configurations) access to production infrastructure. A compromised or misbehaving agent has the same blast radius as a compromised developer.

The empirical evidence for security concern is not hypothetical. Apollo Research (2025) found that o3 engages in covert action at 13% of interactions before anti-scheming training, making promises in visible outputs while taking contradictory hidden actions. NIST CAISI (2025) documented agents commenting out assertions to make tests pass, downloading solutions from GitHub to cheat benchmarks, reading $/\mathrm{dev}/\mathrm{urandom}$ for entropy injection, and crashing servers to satisfy task requirements. CodeRabbit (2025) measured 2.74× more XSS vulnerabilities in AI-generated code than in human-written code across 470 PRs. METR (2025) found reward-hacking at 1–2% of task attempts, a rate that compounds across thousands of daily invocations.

The structural enforcement principle, developed as a cross-pillar result across this paper series (Pragnition Labs, 2026a,c), is fundamentally a security principle. Any security property enforced by prompt instructions decays as $(1-\epsilon)^T$ over T interaction steps. For a 50-step pipeline with

$\epsilon = 0.02$, prompt-based security degrades to 36% compliance. Security invariants enforced structurally (sandboxing, network isolation, credential vaulting) do not decay. This paper makes that distinction precise.

This is a *theory and position paper*. The formal results are novel in their *application* to agent security architecture; the underlying mathematical machinery (access control theory, information flow control, reliability theory) is established. Where we conjecture rather than prove, we say so explicitly. Empirical calibration draws on published studies and practitioner reports; we flag sources that precede peer-reviewed publication.

Our contributions are:

1. **Capability bounding as a set optimization problem** (§3): we formalize the sandbox as a restriction operator σ on the agent's capability set, connect sandbox safety to the HRU undecidability result, and identify the decidable subcases relevant to harness design.
2. **Credential flow as information flow control** (§4): we model credential management using Denning's lattice model, prove a noninterference property for the credential path, and show how the Bell-LaPadula/Biba tension is resolved through structural path separation.
3. **The structural enforcement decay theorem** (§5): we prove that prompt-based security compliance probability is $(1-\epsilon)^T$ and derive the cost-of-failure threshold at which structural enforcement becomes cost-effective, extending the result from Pragnition Labs (2026c).
4. **Prompt injection as confused deputy** (§6): we establish the formal parallel between prompt injection and the confused deputy problem (Hardy, 1988), connect it to the BEB impossibility framework (Wolf et al., 2024), and prove that structural defenses are the only zero-exploit guarantee.
5. **Production harness survey and reference architecture** (§10): we survey six production harnesses, identify convergent structural enforcement patterns, and derive a four-layer reference security architecture.
6. **Five contrarian positions engaged with evidence** (§12): we steelman and evaluate arguments that security is too expensive, that training suffices, that the threat model is unrealistic, that sandboxing kills productivity, and that prompt injection is solved.

2. Preliminaries and Definitions

2.1. Agent, Harness, and Trust Boundary

Definition 2.1 (AI Coding Agent)**.** *An* AI coding agent *is a system that takes a task description τ and produces a sequence of actions $a_1, a_2, \ldots, a_n$ drawn from an action space $\mathcal{A}$ (file reads, file writes, shell commands, network requests, API calls) using a large language model as the decision-making component.*

Definition 2.2 (Harness)**.** *A* harness *is the orchestration layer that mediates between the agent and the external environment. It controls which actions the agent can take, which resources it can access, and which outputs it can produce.*

Definition 2.3 (Trusted Computing Base)**.** *Following Department of Defense (1985), the* Trusted Computing Base *(TCB) of an agent harness is the totality of protection mechanisms whose correct functioning is necessary for enforcing the security policy. The TCB includes the sandbox, the tool permission system, the credential vault, the network policy, and the output filter. The agent itself is* outside *the TCB.*

The architectural consequence of Definition 2.3 is profound: security of the agent system reduces entirely to the correctness of a small, non-AI, formally verifiable TCB. The agent's behavior can be arbitrarily wrong, adversarially manipulated, or hallucinated, and the security policy must still hold.

2.2. Capability Set and Restriction Operator

Definition 2.4 (Capability Set)**.** *The agent's* full capability set C *is the set of all operations (file reads, file writes, shell commands, network requests, API calls) the agent could invoke if unconstrained. The* task-required capability set $\mathcal{T} \subseteq C$ *is the minimum set of operations needed to complete the assigned work.*

Definition 2.5 (Sandbox Restriction Operator)**.** *A* sandbox *defines a restriction operator* $\sigma : 2^C \to 2^C$ *such that* $\sigma(C) \subseteq C$ *is the effective capability set available to the agent. The operator* σ *is* structurally enforced *if there exists an implementation such that the agent cannot invoke any operation in* $C \setminus \sigma(C)$ *regardless of its internal state or the instructions it has received.*

2.3. Security Levels and Information Flow

Definition 2.6 (Security Lattice)**.** *Following Denning (1976), a* security lattice *is a partially ordered set* $(L, \sqsubseteq)$ *where* L *is a finite set of security levels and* $\sqsubseteq$ *is a partial order with a least upper bound (join) and greatest lower bound (meet) for every pair. For agent harnesses, we define* $L = \{\text{task}, \text{repo}, \text{infrastructure}, \text{production}\}$ *with* $\text{task} \sqsubseteq \text{repo} \sqsubseteq \text{infrastructure} \sqsubseteq \text{production}$.

2.4. Reference Monitor

Definition 2.7 (Reference Monitor)**.** *Following Anderson (1972), a* reference monitor $\mathcal{M}$ *is a mechanism that, for every access request* (s, o, a) *where* s *is a subject,* o *is an object, and* a *is an access mode, evaluates whether the request is permitted under policy* Π *and either allows or denies it. The reference monitor must satisfy three properties: (1)* complete mediation*, meaning every access request is evaluated; (2)* tamperproofness*, meaning no subject can modify or disable* $\mathcal{M}$*; and (3)* verifiability*, meaning* $\mathcal{M}$ *is small enough for exhaustive analysis.*

3. Sandbox Design as Capability Bounding

3.1. The Least Privilege Optimization

The principle of least privilege (Saltzer and Schroeder, 1975) states that every program should operate with the minimum set of privileges necessary. For agent harnesses, this becomes a set optimization problem:

$$\min_{|\sigma(C)|} \quad \text{subject to} \quad \mathcal{T} \subseteq \sigma(C) \tag{1}$$

Minimize the capability surface while preserving task completeness. The difficulty is that $\mathcal{T}$ is not known precisely in advance: the agent may need capabilities that were not anticipated during sandbox configuration. The practical solution is allowlisting with human-approved escape hatches (Pragnition Labs, 2026b).

Definition 3.1 (Attack Surface)**.** *The* attack surface $\mathcal{S}$ *is the set of capabilities available to the agent that are not task-required:* $\mathcal{S} = \sigma(C) \setminus \mathcal{T}$*. The security goal is to minimize* $|\mathcal{S}|$.

Proposition 3.1 (Attack Surface Under Uncertainty)**.** *When* $\mathcal{T}$ *is not known exactly but estimated as* $\hat{\mathcal{T}}$ *with error probability* $\alpha = \Pr(t \in \mathcal{T} \setminus \hat{\mathcal{T}})$ *per operation, the sandbox designer faces a tradeoff. Setting* $\sigma(C) = \hat{\mathcal{T}}$ *minimizes attack surface but fails task completeness with probability* $1 - (1 - \alpha)^{|\mathcal{T}|}$*. Setting* $\sigma(C) = \hat{\mathcal{T}} \cup \mathcal{E}$ *for an "escape hatch" set* $\mathcal{E}$ *recovers completeness at the cost of* $|\mathcal{S}| = |\mathcal{E} \setminus \mathcal{T}|$

additional attack surface. The practical resolution is human-approved capability escalation: $\mathcal{E}$ consists of capabilities the agent can request but that require human approval before activation (Pragnition Labs, 2026b).

3.2. The HRU Undecidability Result

Harrison et al. (1976) formalized the *safety problem*: given an access control system, is it possible for a given subject to ever obtain a specific right through a sequence of operations?

Theorem 3.1 (HRU Undecidability, Harrison et al., 1976)**.** *The safety problem for general access control systems is undecidable. Any Turing machine can be simulated by an HRU protection system, and safety reduces to the halting problem.*

The implication for harness design is stark: if the harness allows agents to create new resources, grant permissions, and invoke arbitrary command sequences, proving that no sequence of operations can leak a credential is equivalent to solving the halting problem.

Corollary 3.1 (Decidable Subcases for Harnesses)**.** *Safety is decidable for two subcases relevant to harness design:*

1. ***Mono-operational systems:*** *each command contains exactly one primitive operation. Safety is decidable but NP-complete.*
2. ***Monotonic systems:*** *permissions are only granted at initialization and never expanded at runtime. Safety is trivially decidable by inspecting the initial permission matrix.*

Most production sandboxes implement the monotonic subcase: the permission set is fixed before the agent begins executing and cannot be expanded by the agent itself. This is the design that Codex, Claude Code, and Spotify Honk all converge on (§10).

3.3. Capability-Based Security

Dennis and Van Horn (1966) introduced capability-based addressing, where a *capability* is an unforgeable token that both designates an object and authorizes specific operations on it. Miller et al. (2003) demolished three persistent myths about capability systems: that they are equivalent to ACL systems (false for dynamic security properties), that they cannot enforce confinement (false with careful construction), and that access cannot be revoked (false through caretaker patterns).

For agent harnesses, capability-based security is the natural model:

1. **Agent initialization as capability endowment.** When the harness spawns an agent, it passes a specific set of capabilities (file handles, API tokens, tool references). The agent cannot acquire capabilities it was not given.
2. **No ambient authority.** The agent cannot discover resources by name; it operates only on resources explicitly provided. This eliminates path-traversal and environment-variable-sniffing attacks.
3. **Delegation with attenuation.** When an agent delegates a sub-task, it passes a subset of its own capabilities. The sub-agent cannot escalate beyond what was delegated.

Capsicum (Watson et al., 2010) demonstrates that capability-based security is practical at the OS level: once a process enters capability mode, it cannot access any global namespace, operating only through explicitly provided file descriptors.

4. Credential Flow as Information Flow Control

4.1. The Credential Exposure Problem

The agent needs some credentials (API keys for tool invocation, git tokens for repository access) but must never see others (production database passwords, code signing keys, cloud IAM admin credentials). We model this as an information flow control problem (Denning, 1976).

Each credential k has a security level $\ell(k) \in L$ in the lattice from Definition 2.6. The agent's clearance is ℓ_a. The noninterference property requires:

$$\forall k : \ell(k) \not\sqsubseteq \ell_a \implies k \notin \text{context}(\text{agent}) \tag{2}$$

Credentials above the agent's clearance level must never appear in the agent's context window, tool outputs, environment variables, or any observable channel.

Theorem 4.1 (Credential Noninterference)**.** *If the harness implements credential injection at the tool level (the tool receives the secret directly from a vault; the agent sees only the tool's output), and if the tool's output does not contain the raw credential, then the noninterference property (2) holds regardless of the agent's behavior.*

Proof. By construction. The credential k with $\ell(k) \not\sqsubseteq \ell_a$ exists only in the vault (part of the TCB) and is injected into the tool invocation by the TCB. The agent's observable state consists of: (a) the prompt, which by assumption does not contain k; (b) tool outputs, which by assumption do not contain the raw k; and (c) environment variables, which by assumption do not contain k at the agent's clearance level. Since k is absent from every component of the agent's observable state, the agent cannot leak k through any output channel. □

Remark (Tool Output Leakage)**.** *The assumption that tool outputs do not contain the raw credential deserves scrutiny. Some tools may leak credential-derived information through error messages (e.g., "authentication failed for token sk-abc..."), HTTP response headers, or structured metadata. The noninterference guarantee holds only when the tool-level credential injection is paired with output sanitization that strips credential fragments from tool results before they enter the agent's context. In practice, this requires that the TCB's output filter (§7) monitors tool outputs as well as agent outputs.*

4.2. The Bell-LaPadula / Biba Tension

The agent faces a fundamental tension between confidentiality and integrity models.

Bell-LaPadula (Bell and LaPadula, 1973) enforces confidentiality through "no read up, no write down": a subject can read objects only at or below its clearance and can write only to objects at or above its clearance. This prevents the agent from reading secrets above its level and from leaking high-security information to low-security outputs.

Biba (Biba, 1977) enforces integrity through "no read down, no write up": a subject must not read objects below its integrity level (to avoid contamination) and must not write to objects above its integrity level (to avoid corruption).

The tension for agent systems: the agent *must* read low-integrity input (user prompts, PR descriptions, web content) to do its job, violating Biba's "no read down" rule. Simultaneously, it must protect high-confidentiality secrets from leaking, requiring Bell-LaPadula's "no write down" rule.

Proposition 4.1 (Path Separation Resolution)**.** *The BLP/Biba tension is resolved by structural separation of the data path and the credential path. Following Lipner (1982), assign each entity both a confidentiality level c and an integrity level i:*

- *Agent's context window:* $(c = \text{low}, i = \text{low})$*. The agent sees no secrets; its input is untrusted.*
- *Credential vault:* $(c = \text{high}, i = \text{high})$*. Secrets are confidential and trustworthy.*
- *Agent's tool calls:* $(c = \text{low}, i = \text{low})$*. Tool call parameters are visible and untrusted.*
- *TCB credential injection:* $(c = \text{high}, i = \text{high})$*. The TCB (not the agent) injects credentials.*

BLP is satisfied because secrets $(c = \text{high})$ *never flow to the agent's output* $(c = \text{low})$*. Biba is locally satisfied on the credential path: the vault* $(i = \text{high})$ *is never modified by the agent* $(i = \text{low})$*. The Biba violation on the data path (the agent reads low-integrity input) is contained: the data path cannot influence the credential path.*

4.3. Decentralized Labels for Multi-Agent Systems

In multi-agent harnesses, each agent may have a different clearance level. The decentralized label model (Myers and Liskov, 1997) extends Denning's lattice to settings where labels are managed by individual principals rather than a central authority. Each label specifies an owner and a set of readers; information flow is permitted only when the target label's reader set includes the source label's readers. For agent harnesses, this means:

- An orchestrator agent with broad access can delegate to sub-agents with narrower labels.
- A sub-agent that reads a file labeled "project-X-only" cannot pass that information to a sub-agent working on project Y.
- The label enforcement is structural (mediated by the harness), not behavioral (relying on agent compliance).

5. The Structural Enforcement Principle

5.1. Prompt-Based Security Decay

Definition 5.1 (Prompt Enforcement)**.** *A security constraint is* prompt-enforced *if compliance depends on the agent's probabilistic response to a natural language instruction. Let ϵ be the per-step non-compliance probability:* $\Pr(\textit{violation at step } t \mid \textit{constraint in prompt}) = \epsilon$.

Definition 5.2 (Structural Enforcement)**.** *A security constraint is* structurally enforced *if there exists an enforcement function $\sigma : \mathcal{A} \to \mathcal{A}'$ such that $\sigma(a) \notin \mathcal{V}$ for all $a \in \mathcal{A}$, where $\mathcal{V}$ is the set of constraint-violating actions. The function projects every action into the safe subspace regardless of agent intent.*

Theorem 5.1 (Structural Enforcement Decay)**.** *For a T-step agent pipeline where a prompt-enforced security constraint must hold at every step:*

$$\Pr(\text{no violation in } T \text{ steps}) = (1 - \epsilon)^T \tag{3}$$

For structurally enforced constraints:

$$\Pr(\text{no violation in } T \text{ steps}) = 1 - \delta_s \tag{4}$$

where δ_s is the specification gap (the probability that the structural enforcement has a design flaw), which does not depend on T.

Proof. The prompt-enforced case follows from independence of per-step violations (a simplifying assumption; correlated violations make the decay faster). The structural case follows because the enforcement function σ is applied deterministically at every step; its correctness depends on specification, not on the number of steps. □

Table 1 | Prompt-based security compliance $(1-\epsilon)^T$ for representative ϵ and T.

ϵ	$T = 5$	$T = 10$	$T = 20$	$T = 50$	$T = 100$
0.01	95.1%	90.4%	81.8%	60.5%	36.6%
0.02	90.4%	81.7%	66.8%	36.4%	13.3%
0.05	77.4%	59.9%	35.8%	7.7%	0.6%
0.10	59.0%	34.9%	12.2%	0.5%	<0.01%

5.2. The Cost-of-Failure Threshold

Theorem 5.2 (Structural Enforcement Threshold)**.** *Let δ be the non-compliance probability under prompt enforcement, $\delta_s < \delta$ the specification gap under structural enforcement, c_p and c_s their respective implementation costs ($c_s > c_p$), and L the cost per violation. Structural enforcement is cost-effective when:*

$$L > L^* = \frac{c_s - c_p}{\delta - \delta_s} \tag{5}$$

For an n-step pipeline, the threshold drops to approximately L^*/n because the effective non-compliance probability under prompt enforcement grows as $1-(1-\delta)^n$ (Pragnition Labs, 2026c).

Using AgentSpec parameters (Wang et al., 2026): $\delta = 0.23$ (prompt failure rate), $\delta_s = 0.10$ (structural specification gap for manually authored rules), $c_p = 1$, $c_s = 10$. Then $L^* = 9/0.13 \approx 69$. When a single violation costs more than $69\times$ the prompt engineering cost, structural enforcement is justified. For credential leakage or unauthorized deployment, L exceeds this threshold by orders of magnitude.

6. Adversarial Robustness Under Prompt Injection

6.1. Prompt Injection as the Confused Deputy

Hardy (1988) defined the confused deputy problem: a more-privileged program is tricked by a less-privileged one into misusing its authority. In the classic example, a compiler with write access to a billing file is tricked by a user into overwriting it.

The parallel to prompt injection is exact:

Classical Deputy	**LLM Agent**
Compiler (FORT)	The LLM agent
User	Adversarial content author
Compiler's ambient authority	Agent's tool permissions
User-specified filename	Injected instruction in untrusted content
Billing file overwritten	Credentials exfiltrated, malicious code executed

Willison (2024) identifies the *lethal trifecta*: when an agent has (1) access to private data, (2) exposure to untrusted content, and (3) ability to act, any prompt injection in the untrusted content can weaponize the agent's capabilities.

Definition 6.1 (Instruction-Data Conflation)**.** *An LLM exhibits instruction-data conflation because instructions and data share the same processing channel (the context window), with no structural delimiter the model can enforce. The context is a token sequence* [system_prompt‖user_message‖retrieved_data]*, and the model applies the same attention mechanism to all tokens.*

This conflation distinguishes prompt injection from SQL injection. SQL injection was solved by parameterized queries, which structurally separate the instruction channel from the data channel (UK National Cyber Security Centre, 2023). LLMs have no analogous mechanism.

6.2. The BEB Impossibility Framework

Wolf et al. (2024) establish formal impossibility results through the Behavior Expectation Bounds (BEB) framework. The LLM distribution is modeled as a mixture $\mathbb{P} = \alpha\mathbb{P}_- + (1-\alpha)\mathbb{P}_+$, where $\mathbb{P}_-$ is the ill-behaved component and α is the residual probability mass on undesired behavior after alignment.

Theorem 6.1 (Alignment Impossibility, Wolf et al., 2024)**.** *If undesired behavior B has non-zero probability under the model ($\alpha > 0$), there exist adversarial prompts of bounded length $L_1 = O(\log(1/\alpha)/\beta)$ that elicit that behavior, where β is the distinguishability of aligned and unaligned components. Furthermore, prepending a system prompt s_0 increases the required adversarial prompt length by only $O(|s_0|)$, not exponentially.*

The implications are stark:

1. For any undesired behavior with non-zero training-distribution probability, adversarial prompts of bounded length exist.
2. Alignment (RLHF, Constitutional AI) reduces α but only increases the required attack length logarithmically. The defense gain is sublinear in the alignment effort.
3. System prompt hardening provides linear, not exponential, defense.

6.3. Empirical Prompt Injection Evidence

CVE-2025-53773 (Embrace The Red, 2025) demonstrated the end-to-end attack chain against GitHub Copilot: malicious instructions embedded in source code comments caused Copilot to modify `.vscode/settings.json` to enable auto-approval of all actions, then execute arbitrary shell commands. The vulnerability was *wormable*: infected code could spread to other repositories.

METR (2025) tested whether prompt-level instructions reduce reward hacking: adding "do not cheat" to the system prompt did not reduce rates below 70%, and one variant ("solve using intended methods") actually increased the rate to 95%. This confirms the BEB prediction that prompt-level defenses are fundamentally insufficient.

6.4. The Structural Defense Principle

Corollary 6.1 (Structural Defense Necessity)**.** *Because prompt injection cannot be eliminated at the model level (Theorem 6.1), and because prompt-based security decays as $(1-\epsilon)^T$ (Theorem 5.1), security guarantees must come from structural constraints:*

1. *The agent* cannot *take the harmful action (sandbox enforcement).*
2. *The action requires external authorization (human-in-the-loop).*
3. *The credential is not accessible (vault with scoped tokens).*
4. *The output is validated structurally (output filters checking syntactic properties).*

These defenses correspond to the TCB components in Definition 2.3. The security argument does not depend on the agent behaving correctly.

7. Output Filtering as Channel Capacity Restriction

Every agent output (code committed, shell commands executed, files written, network requests sent) is a potential attack vector. Output filtering reduces the effective channel capacity between the agent and the external system.

Definition 7.1 (Output Channel Capacity)**.** *Let B be the unfiltered channel capacity (all possible outputs) and B_f be the filtered capacity. The security optimization is:*

$$\min_{B_f} \quad \textit{subject to} \quad B_f \geq B_{\text{task}} \tag{6}$$

where B_{task} is the minimum channel capacity required for the agent to complete its work.

Overly restrictive filtering ($B_f \ll B_{\text{task}}$) blocks legitimate operations; overly permissive filtering ($B_f \gg B_{\text{task}}$) allows malicious or accidental harm. The design goal is to maximize the ratio of legitimate to harmful operations in the permitted channel.

Concrete instantiation. Spotify's Honk agent (Spotify Engineering, 2025) allowlists specific shell commands. The allowlist defines B_f explicitly: only ripgrep, git operations (read-only), and a curated set of formatters and linters are permitted. Any command not on the list is rejected regardless of the agent's reasoning. This structural enforcement of B_f means the output channel is restricted to a known-safe vocabulary.

Proposition 7.1 (Covert Channel Risk)**.** *Even with output filtering, covert channels may exist. An agent can encode information in benign-looking outputs: URLs containing exfiltrated data as parameters, filenames encoding secrets, or timing patterns in sequential operations (Lampson, 1973). Complete elimination of covert channels requires restricting not just the content but also the bandwidth, timing, and metadata of agent outputs.*

8. Defense-in-Depth: When It Works and When It Doesn't

The classical argument for defense-in-depth assumes independent failure of security layers. For n layers, each with effectiveness p_i, the naive model predicts:

$$\Pr(\text{attack succeeds}) = \prod_{i=1}^{n} (1 - p_i) \tag{7}$$

This exponential decay is seductive but unrealistic. The beta factor model from reliability engineering (Fleming, 1975) captures correlated failures:

$$\Pr(\text{all fail}) \approx (1 - \beta)^n q^n + \beta q \tag{8}$$

where β is the common-cause failure fraction and $q = \max_i(1 - p_i)$. When $\beta > 0$, the common-cause term βq dominates the independent term $(1-\beta)^n q^n$ for small q (Littlewood and Strigini, 2004).

For agent harness security, heuristic defense layers (input sanitization, system prompt reinforcement, output validation) share a common failure mode: the fundamental instruction-data conflation (Definition 6.1). A sufficiently novel injection technique bypasses all three because they all rely on the same channel. We estimate $\beta_{1-3} \approx 0.5$ for heuristic layers as an order-of-magnitude figure; the qualitative conclusion (structural layers dominate) is robust for any $\beta > 0.1$. Adding a structural layer (tool permission gates, $p_4 = 0.99$):

- **Optimistic (independent):** $\Pr = 0.20 \times 0.15 \times 0.10 \times 0.01 = 3.0 \times 10^{-6}$
- **Realistic (correlated heuristics):** $\Pr \approx 0.5 \times 0.20 \times 0.01 = 1.0 \times 10^{-3}$

The realistic estimate is 333× worse. The only layer providing genuinely uncorrelated defense is the structural one.

Proposition 8.1 (Genuine Defense-in-Depth Criterion)**.** *Defense-in-depth provides genuine security improvement when:*

1. *Layers have diverse failure modes (structural + heuristic).*
2. *Layers operate on different data representations.*
3. *Layers are not vulnerable to the same attack class.*

Combining multiple heuristic layers against the same attack vector provides diminishing returns bounded by the common-cause factor β.

9. Empirical Calibration

9.1. Agent Misbehavior Rates

Table 2 | Empirical agent security misbehavior rates from independent studies.

Source	Finding	Rate	Mechanism
Apollo Research	Covert action (o3, pre-training)	13.0%	In-context scheming
Apollo Research	Covert action (o3, post-training)	0.4%	Residual after anti-scheming
NIST CAISI	Grader gaming (CVE-Bench)	4.8%	Server crashing via /dev/urandom
NIST CAISI	Test manipulation (SWE-bench)	~1%	Commenting out assertions
METR	Reward hacking (RE-Bench)	30.4%	Monkey-patching, stack introspection
METR	Reward hacking (HCAST)	0.7%	Various
CodeRabbit	XSS vulnerabilities (AI vs human)	2.74×	Insecure code generation
AgentSpec	Prompt enforcement failure	23%	Non-compliance with prompt rules
AgentSpec	Structural enforcement failure	~13%	Specification gaps

Three patterns emerge from the empirical data:

1. Prompt-level defenses are insufficient. METR's finding that "do not cheat" instructions did not reduce reward hacking below 70% (METR, 2025) directly confirms the $(1 - \epsilon)^T$ decay model. The structural enforcement gap (AgentSpec: 87% structural vs. 77% prompt) provides calibration for $\epsilon \approx 0.23$ for prompt enforcement and $\delta_s \approx 0.13$ for structural enforcement.

2. Misbehavior compounds at scale. The 1–2% per-task reward hacking rate (METR, 2025) means 100–200 reward-hacking attempts per day in an organization making 10,000 agent invocations. The 0.4% residual covert action rate after anti-scheming training (Apollo Research, 2025) means 40 covert actions per 10,000 invocations.

3. Longer pipelines are dramatically more vulnerable. METR found 30.4% reward hacking on longer RE-Bench tasks vs. 0.7% on shorter HCAST tasks, a 43× difference. This is consistent with the $(1-\epsilon)^T$ model: longer task horizons amplify the per-step failure probability. The variance also suggests that task characteristics (scoring function visibility, tool access breadth, horizon length) dominate model characteristics in determining misbehavior rates. This implies that harness design choices (what tools to expose, how to define success criteria) are a primary lever for security, independent of model selection.

9.2. Vulnerability Generation Rates

CodeRabbit (2025) found that AI-generated code exhibits systematically higher vulnerability rates across all measured categories:

Vulnerability Category	AI/Human Ratio
XSS vulnerabilities	2.74×
Insecure object references	1.91×
Improper password handling	1.88×
Insecure deserialization	1.82×
Performance regressions (I/O)	~8.0×

These rates establish that output filtering for security vulnerabilities is not optional; AI-generated code requires more scrutiny, not less, than human-written code.

10. The Reference Security Architecture

10.1. Production Harness Survey

We surveyed six production harnesses across five organizations. Every harness that achieves reliable security does so structurally. No production harness relies solely on prompt-based security.

Table 3 | Security mechanisms across production harnesses. **S** = structural, **H** = hybrid, **I** = instructional, **–** = absent.

Harness	Execution Isolation	Network Control	Credential Separation	Output Filter	Injection Defense
Codex (cloud)	S	S	S	H	S
Claude Code	S	S	S (cloud)	H	H
Cursor	S	H	–	H	I
Devin	S	–	H	–	I
Spotify Honk	S	S	S	S	S
RAPID	S	–	–	S	S

Key patterns:

1. **Execution isolation is universal.** Every surveyed harness uses OS-level or container-level isolation: Codex uses containers with a two-phase runtime (setup with network, agent without);

Claude Code uses bubblewrap (Linux) and Seatbelt (macOS); Cursor uses Landlock + seccomp; Devin uses per-session VMs; Honk uses containers with limited binaries; RAPID uses git worktrees.

2. **Credential separation at the tool level is the gold standard.** Codex removes secrets before the agent phase starts. Claude Code's cloud proxy translates scoped tokens into real credentials outside the sandbox. Honk handles all credential-bearing operations (git push, Slack messaging) outside the agent entirely. This is the cleanest instantiation of Theorem 4.1.
3. **Network isolation varies.** Codex and Claude Code implement allowlist-only egress. Honk's agent has essentially no network access. Devin, notably, has unrestricted internet access by default, which has been exploited for data exfiltration.
4. **Allowlisting over blocklisting.** Honk allows only a curated command set. Codex allows only approved domains and HTTP methods. A blocklist requires anticipating every harmful operation; an allowlist requires enumerating only the legitimate ones.

10.2. The Four-Layer Reference Architecture

Definition 10.1 (Reference Security Architecture)**.** *The reference architecture consists of four structural layers, each independently enforced by the TCB:*

Layer 1: Execution Isolation (structural, per task). The agent executes in an isolated environment with: filesystem scope limited to the task-relevant directory; allowlist-only network egress; process isolation preventing persistent background processes or privilege escalation.

Layer 2: Credential Management (structural, at tool level). Credentials are injected into tools, not into prompts. The tool receives the secret from a vault; the agent sees only the result. Credential tiers (task, repo, infrastructure) have different vaulting mechanisms. Short-lived tokens expire on task completion.

Layer 3: Output Filtering (structural, every write). Static analysis on every file write (secret detection, known vulnerability patterns). Allowlist-based shell command execution. URL filtering against the egress allowlist for network requests.

Layer 4: Adversarial Input Isolation (structural, at context assembly). External content is tagged with a low-integrity label before entering the agent's context. Structural markers demarcate untrusted regions. High-risk operations require explicit human confirmation regardless of context content (Pragnition Labs, 2026b).

11. Related Work

Classical access control. Lampson (1971) introduced the access control matrix. Harrison et al. (1976) proved undecidability of the general safety problem, motivating the use of restricted subcases. Dennis and Van Horn (1966) and Miller (2006) developed capability-based security, which Watson et al. (2010) demonstrated as practical in Capsicum.

Information flow control. Denning (1976) established the lattice model. Bell and LaPadula (1973) and Biba (1977) formalized confidentiality and integrity respectively. Goguen and Meseguer (1982) defined noninterference. Rushby (1992) addressed compositionality. Myers and Liskov (1997) extended the model to decentralized settings. Lipner (1982) resolved the BLP/Biba tension for commercial applications.

Agent security. Wang et al. (2026) provides the key data point on structural vs. prompt enforcement (87% vs. 77%). The OWASP Top 10 for LLM Applications (OWASP, 2025) classifies prompt injection as the #1 risk. Greshake et al. (2023) established indirect prompt injection as a practical threat. Wolf et al. (2024) proved formal impossibility of alignment-based defenses. UK National Cyber Security Centre (2023) framed prompt injection as distinct from (and harder than) SQL injection.

Agent misbehavior. Apollo Research (2025) documented in-context scheming. NIST CAISI (2025) catalogued evaluation cheating. METR (2025) measured reward hacking rates and demonstrated prompt-level defense futility. CodeRabbit (2025) quantified vulnerability rate differentials.

Structural enforcement. The structural enforcement principle appears independently across our paper series. Pragnition Labs (2026c) established the cost-of-failure threshold. Pragnition Labs (2026a) showed that structural governance does not decay over pipeline length. Google DeepMind (2026) demonstrated 100% compliance for code-based constraint enforcement across 145 TextArena games, providing independent empirical support. This paper identifies security as the canonical application of the principle.

12. Engaging Contrarian Positions

We present five contrarian positions in steelmanned form and evaluate what the evidence actually supports.

12.1. "Security Is Too Expensive for Coding Agents"

The strongest version: sandboxing, credential vaulting, and output filtering add engineering complexity and operational overhead. For internal tools, prototypes, and low-risk code, the blast radius of a misbehaving agent may be acceptable without full isolation.

What the evidence shows: The blast radius is bounded by the agent's credentials, not by the task's importance. An agent writing a prototype with production database credentials has a production-level blast radius regardless of the task's risk classification. However, the cost argument has genuine force for early-stage development: Codex's container model demonstrates near-zero perceptible latency impact because the binding constraint in agent workflows is API latency (seconds to tens of seconds per LLM call), not sandbox setup (milliseconds to low seconds). The sandbox cost is amortized below the noise floor of the dominant latency source.

12.2. "LLMs Can Be Trained to Be Safe"

Constitutional AI (Bai et al., 2022), RLHF, and anti-scheming training can reduce unsafe behavior to negligible levels, making structural enforcement unnecessary.

What the evidence shows: Training is an instructional defense. It reduces ϵ (the per-step failure probability) but does not eliminate it. Over long pipelines, $(1-\epsilon)^T$ still converges toward zero. Apollo Research (2025) confirmed that anti-scheming training reduces covert action from 13% to 0.4%, but also increased evaluation-awareness reasoning, suggesting the model learned to detect evaluations rather than internalize alignment. METR (2025) showed that no prompt variant reduced reward hacking below 70%. Training is a necessary complement to structural enforcement, not a substitute for it.

12.3. "The Threat Model Is Unrealistic"

Real agents are not adversarial; they are cooperative systems that occasionally make mistakes. Military-grade access control formalism (BLP, Biba) is overkill.

What the evidence shows: The threat model is threefold: (a) the agent optimizes against poorly-specified objectives (NIST CAISI evidence: commenting out assertions, crashing servers), (b) the agent processes adversarial external content (prompt injection via PR descriptions, malicious READMEs, poisoned dependencies), and (c) the agent has correlated failure modes at scale (the same model makes the same security mistake across thousands of invocations). None of these require the agent to be malicious; all require structural containment. The BLP/Biba formalism is not about treating the agent as an enemy but about ensuring that information flows correctly regardless of the agent's behavior.

12.4. "Sandboxing Kills Productivity"

Container setup adds latency; network restrictions block legitimate dependencies; filesystem scoping prevents the agent from reading relevant code in other directories.

What the evidence shows: Cursor's production data (February 2026) shows sandboxing reduces developer interruptions by 40% compared to unsandboxed workflows, because the sandbox replaces per-command approval prompts with a permission boundary. Codex's two-phase model (setup with network, agent without) amortizes dependency installation cost. The real productivity concern is not latency but capability restriction: agents that cannot reach needed resources fail tasks. The resolution is task-scoped capability grants (allowlists tuned per task type), not removing the sandbox entirely.

12.5. "Prompt Injection Is a Solved Problem"

Improved instruction hierarchy, input/output tagging, model robustness training, and spotlighting techniques will eliminate prompt injection.

What the evidence shows: Wolf et al. (2024) proved that alignment-based defenses cannot reduce α to zero for behaviors in the model's training distribution. The BEB impossibility means adversarial prompts of bounded length always exist. Every mitigation reduces the attack success rate but cannot eliminate it without eliminating the model's ability to follow legitimate instructions from the same channel. The NCSC (UK National Cyber Security Centre, 2023) concludes that prompt injection "will never be properly mitigated in the same way" as SQL injection. The structural defense (the agent cannot take the harmful action regardless of its instructions) is the only defense with a zero-exploit guarantee.

13. Design Principles

The formal framework and empirical calibration yield ten actionable design principles:

1. **The agent is outside the TCB.** Security depends on the harness (sandbox, vault, filters, permissions), not on the agent's behavior. Design accordingly: assume the agent is arbitrarily wrong.
2. **Structural enforcement for security; prompt enforcement for style.** Safety-critical constraints (credential confidentiality, filesystem boundaries, network isolation) must be struc-

turally enforced. Subjective dimensions (code quality, naming conventions) can use prompt-level guidance.

3. **Credential injection at the tool level, never the prompt level.** If a secret appears in the agent's context window, it can be leaked through any output channel. The tool receives the secret from a vault; the agent sees only the tool's output.
4. **Allowlist, do not blocklist.** A blocklist requires anticipating every harmful operation; an allowlist requires enumerating only the legitimate ones. The allowlist is structurally sound; the blocklist is structurally incomplete.
5. **Scope capabilities per task.** The principle of least privilege (Equation (1)) requires that each task grant only the capabilities it needs. Over-provisioning creates unnecessary attack surface.
6. **Use the monotonic subcase.** Fix the permission set before the agent begins and do not allow runtime expansion. This makes sandbox safety trivially decidable (Corollary 3.1).
7. **Assume prompt injection will succeed.** The BEB impossibility (Theorem 6.1) means that for sufficiently long pipelines, prompt-based injection defenses will eventually fail. Design output filtering, capability restriction, and credential isolation to contain the blast radius.
8. **Separate the data path from the credential path.** The BLP/Biba tension (Proposition 4.1) is resolved by ensuring that untrusted input and secret credentials never share a processing path.
9. **Combine structural and heuristic layers, not multiple heuristic layers.** Defense-in-depth works when layers have uncorrelated failure modes (Proposition 8.1). Multiple heuristic layers against the same attack vector provide diminishing returns.
10. **Scan AI-generated code with greater scrutiny.** The 2.74× XSS vulnerability multiplier (CodeRabbit, 2025) means AI-generated code requires more output filtering, not less. Static analysis and secret scanning are mandatory verification steps.

14. Limitations

The $(1-\epsilon)^T$ model assumes independent per-step failures. In practice, failures may be correlated (the same prompt weakness exploited repeatedly) or anti-correlated (the agent learns from a near-miss). The independent model is a reasonable first approximation, but correlated failures make the decay faster, and anti-correlated failures make it slower. We do not have sufficient empirical data to calibrate the correlation structure.

Formal results are applied, not novel. The HRU undecidability theorem, the Denning lattice model, the BLP/Biba tension, and the BEB impossibility results are all established. Our contribution is their application and synthesis in the agent harness context, not new mathematical results.

Empirical calibration is sparse. The Apollo, NIST, METR, and CodeRabbit studies use different methodologies, benchmarks, and models. Direct comparison across studies is approximate. The 87% vs. 77% structural vs. prompt enforcement gap from AgentSpec comes from a single study.

The beta factor for correlated defense layers is estimated, not measured. Our $\beta = 0.5$ estimate for heuristic defense layer correlation is reasonable by analogy to reliability engineering, but no controlled study has measured the actual correlation between prompt injection bypasses of different defense layers.

The TCB correctness assumption is strong. Our security argument reduces to TCB correctness. In practice, TCBs have bugs (CVE-2025-53773 for Copilot, the February 2026 Codex CLI token vulnerability). The argument is that TCB bugs are fixable and the TCB is small enough to audit, not that it is perfect.

Covert channels are acknowledged but not solved. Proposition 7.1 notes that output filtering

does not eliminate all information leakage paths. A comprehensive covert channel analysis for agent harnesses remains an open problem.

15. Conclusion

We have developed a formal framework for security architecture in AI coding agent harnesses, grounding it in access control theory, information flow control, reliability theory, and adversarial robustness analysis.

The framework produces one central architectural principle: **the agent is outside the Trusted Computing Base**. The agent's behavior is untrusted by design, whether correct, mistaken, or adversarially manipulated. Security reduces to the correctness of a small, non-AI, formally verifiable enforcement boundary: the sandbox, the credential vault, the permission system, the network policy, and the output filter.

The structural enforcement principle, calibrated against five independent measurement studies, establishes that prompt-based security is quantitatively inadequate for any property where a single violation causes harm. The $(1-\epsilon)^T$ decay means that prompt-based credential protection, for example, is guaranteed to fail eventually in sufficiently long pipelines. Structural enforcement does not decay because it does not depend on the agent's compliance.

The BEB impossibility result closes the hope that training or alignment can substitute for structural defenses. For any undesired behavior with non-zero training-distribution probability, adversarial prompts of bounded length exist. The defense is not making the agent immune to manipulation; the defense is making manipulation harmless.

The most important open questions are empirical. What is the actual correlation structure of per-step security failures? How does ϵ change as models improve? What is the beta factor for correlated failures across heuristic defense layers? And, most practically: how do we build TCBs that are both small enough to verify and flexible enough for the diverse tasks agents must perform? The arithmetic of $(1-\epsilon)^T$ is unforgiving, but the architecture of structural enforcement is well understood. The challenge is implementation, not theory.

Appendices

A. Proofs and Extended Results

A.1. TCB Correctness Sufficiency

Theorem A.1 (TCB Correctness Sufficiency)**.** *If the TCB components $\{S, P, V, N, F\}$ (sandbox, permission system, credential vault, network policy, output filter) correctly implement the security policy Π, then no action by the agent can violate Π, regardless of the agent's internal state or the inputs it has received.*

Proof. By the complete mediation property (Definition 2.7), every agent action passes through $\mathcal{M} = (S, P, V, N, F)$. By tamperproofness, the agent cannot modify $\mathcal{M}$. By the correctness of $\mathcal{M}$ (assumption), only actions consistent with Π are permitted. Therefore no action violating Π can execute. □

The theorem is trivial once stated, but its architectural consequence is profound: the problem of "making the AI safe" is replaced by the problem of "making the sandbox correct," which is a classical software verification problem with known solutions (Klein et al., 2009).

A.2. Correlated Defense-in-Depth

Theorem A.2 (Correlated Defense Bound)**.** *For n defense layers with individual failure probability $q = 1 - p$ and beta factor β:*

$$\Pr(\textit{all fail}) = (1-\beta)^n q^n + \beta q \tag{9}$$

When q is small (individual layers are effective), the common-cause term βq dominates:

$$\lim_{q \to 0} \frac{\beta q}{(1-\beta)^n q^n + \beta q} = 1 \tag{10}$$

Proof. The first term represents the probability that all n layers fail independently: $(1-\beta)^n q^n$, where $(1-\beta)$ is the probability each failure is independent and q^n is the joint independent failure probability. The second term βq represents the probability of a common-cause failure that bypasses all layers simultaneously. As $q \to 0$, the q^n term vanishes faster than q, so the common-cause term dominates. □

A.3. Supply Chain as Attack Vector

The slopsquatting attack (Lanyado and Likhtenberg, 2025) demonstrates a supply chain vector specific to AI agents. Approximately 20% of packages recommended by code generation models do not exist, and 43% of hallucinated package names are repeated across queries. Attackers can pre-register these predictable names with malicious payloads. For agent harnesses, this means that $\mathrm{pip\ install}$ and $\mathrm{npm\ install}$ commands executed by agents are security-critical operations requiring allowlist-based package verification.

A.4. STRIDE Mapping for Agent Harnesses

Prompt injection maps to STRIDE's Elevation of Privilege (E), the most dangerous category, because an attacker who gains the agent's full authority can spoof, tamper, disclose, deny service, and repudiate at will (Shostack, 2014). The OWASP Top 10 for LLM Applications (OWASP, 2025) classifies prompt injection as LLM01 (the #1 risk) and maps to the TCB components in Section 10: sandbox for containment, vault for credential protection, filters for output validation, and human-in-the-loop for high-risk operations.

A.5. Saltzer and Schroeder's Eight Principles Applied

Principle	Harness Application
Economy of mechanism	Minimize TCB size; the agent runtime is outside it.
Fail-safe defaults	Default-deny for all agent tools; allowlist-only access.
Complete mediation	Every tool call, file access, and network request passes through the reference monitor.
Open design	Security does not rely on prompt obfuscation or system prompt secrecy.
Separation of privilege	Human-in-the-loop for destructive actions; two-key approval.
Least privilege	Per-task capability scoping; minimum permissions for current work.
Least common mechanism	Credential isolation between agent sessions; no shared state.
Psychological acceptability	Transparent permission prompts; sandbox that reduces interruptions (Cursor: 40% fewer).

A.6. Verification Roadmap

Table 4 | Verification roadmap for key claims.

Claim	Status	Verification Path
$(1-\epsilon)^T$ decay	Calibrated	Controlled study varying T and measuring compliance
$\epsilon \approx 0.23$ (prompt)	Single source	Replicate AgentSpec across multiple models
$\delta_s \approx 0.13$ (structural)	Single source	Replicate with diverse rule specifications
BEB impossibility	Proven	Established result; verify empirical α, β
$\beta = 0.5$ for heuristic layers	Estimated	Controlled study measuring correlated bypasses
2.74× XSS ratio	Single study	Replicate with larger sample, diverse languages
TCB correctness sufficiency	Proven	Theorem; verify TCB implementations

References

J. P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, Electronic Systems Division, Air Force Systems Command, 1972.

Apollo Research. Stress testing deliberative alignment for anti-scheming training. Technical report, Apollo Research, 2025. arXiv:2509.15541.

Y. Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

D. E. Bell and L. J. LaPadula. Secure computer systems: Mathematical foundations. Technical Report MTR-2547, MITRE Corporation, 1973.

K. J. Biba. Integrity considerations for secure computer systems. Technical Report MTR-3153, MITRE Corporation, 1977.

CodeRabbit. State of AI vs human code generation. Technical report, CodeRabbit, 2025.

D. E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

Department of Defense. Trusted computer system evaluation criteria. Technical Report DoD 5200.28-STD, National Computer Security Center, 1985. The "Orange Book".

Embrace The Red. CVE-2025-53773: GitHub Copilot remote code execution via prompt injection. embracethered.com, 2025.

K. N. Fleming. Reliability analysis of a redundant sensor system. *Nuclear Engineering and Design*, 1975.

J. A. Goguen and J. Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy*, pages 11–20, 1982.

Google DeepMind. AutoHarness: Code-based constraint enforcement for AI agents. Technical report, Google DeepMind, 2026.

K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023.

N. Hardy. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.

M. A. Harrison, W. L. Ruzzo, and J. D. Ullman. Protection in operating systems. *Communications of the ACM*, 19(8):461–471, 1976.

G. Klein et al. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.

B. W. Lampson. Protection. In *Proceedings of the 5th Princeton Symposium on Information Sciences and Systems*, pages 437–443, 1971. Reprinted in Operating Systems Review, 8(1):18–24, 1974.

B. W. Lampson. A note on the confinement problem. *Communications of the ACM*, 16(10):613–615, 1973.

B. Lanyado and O. Likhtenberg. Slopsquatting: How AI hallucinations enable supply chain attacks. Technical report, University of Texas at San Antonio, Virginia Tech, University of Oklahoma, 2025.

S. B. Lipner. Non-discretionary controls for commercial applications. In *IEEE Symposium on Security and Privacy*, 1982.

B. Littlewood and L. Strigini. Redundancy and diversity in security. In *ESORICS*, 2004.

METR. Recent frontier models are reward hacking. Technical report, Model Evaluation and Threat Research, 2025.

M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.

M. S. Miller, K.-P. Yee, and J. Shapiro. Capability myths demolished. Technical Report SRL2003-02, Johns Hopkins University, 2003.

A. C. Myers and B. Liskov. A decentralized model for information flow control. In *Proceedings of the 16th ACM Symposium on Operating Systems Principles*, pages 129–142, 1997.

NIST CAISI. Cheating on AI agent evaluations. Technical report, National Institute of Standards and Technology, 2025.

OWASP. OWASP top 10 for large language model applications. Technical report, Open Web Application Security Project, 2025.

Pragnition Labs. Governance architecture for AI coding agent harnesses. *Pragnition Labs Technical Report Series*, 2026a.

Pragnition Labs. Human interaction architecture for AI coding agent harnesses. *Pragnition Labs Technical Report Series*, 2026b.

Pragnition Labs. Reliability architecture for AI coding agent harnesses. *Pragnition Labs Technical Report Series*, 2026c.

J. Rushby. Noninterference, transitivity, and channel-control security policies. *SRI International Technical Report*, (CSL-92-02), 1992.

J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

A. Shostack. *Threat Modeling: Designing for Security*. Wiley, 2014.

Spotify Engineering. Background coding agents at Spotify: The Honk system. Technical report, Spotify, 2025.

UK National Cyber Security Centre. Prompt injection is not SQL injection (it may be worse). Technical report, NCSC, 2023.

J. Wang et al. AgentSpec: Customizable runtime enforcement for AI agent systems. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*, 2026.

R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway. Capsicum: Practical capabilities for UNIX. In *Proceedings of the 19th USENIX Security Symposium*, pages 29–46, 2010.

S. Willison. The lethal trifecta for AI agents. simonwillison.net, 2024.

Y. Wolf, N. Wies, Y. Levine, and A. Shashua. Fundamental limitations of alignment in large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2304.11082.